CC-RRT Implementation Project

Pranay Katyal Ethan Zhong Manav Mepani Harsh Chhajed

***Abstract*—Autonomous robots are increasingly expected to navigate complex, uncertain environments in applications ranging from self-driving cars to warehouse automation. Ensuring both safety and efficiency under uncertainty remains a formidable challenge. In this work, we present our implementation of the Chance-Constrained Rapidly-Exploring Random Tree (CC-RRT) algorithm, based on the formulation by [1], which integrates probabilistic constraints directly into sampling- based planning to guarantee collision risk below a user-defined threshold.**

Built within the Open Motion Planning Library (OMPL) in C++, our planner propagates Gaussian state distributions throughout the RRT expansion process and performs fast probabilistic collision checks, similar in spirit to the earlier probabilistic path planning work by [2]. We benchmark our implementation in a prototypical 2D workspace featuring convex obstacles, evaluating path length, probabilistic safety (p_{safe}), runtime, and success rate over numerous trials.

Our results demonstrate that CC-RRT achieves real-time performance with tunable safety margins, replicating key behaviors from the foundational literature. Readers will find detailed methodology in Section II, implementation insights in Section II-B, and experimental analysis in Section IV.

I. Introduction

Autonomous robots are increasingly deployed in complex environments where uncertainty arises from sensor noise, actuator disturbances, and unpredictable obstacle motion. Traditional sampling-based planners such as Rapidly-Exploring Random Trees (RRT) [3] excel at efficient exploration but rely on deterministic collision checks, rendering them unsafe under stochastic disturbances or incomplete world models. A common remedy tightening obstacle boundaries or inflating robot footprints induces excessive conservatism or demands expensive mixed integer optimizations, compromising real-time performance.

Chance constraints offer a principled alternative by bounding the probability of constraint violation below a user-defined threshold $1 - p_{\text{safe}}$ through reasoning over state distributions instead of point estimates [2]. However, classical chance-constrained formulations for linear systems with Gaussian noise lead to costly nonlinear or integer programs. Lüders et al. introduced the Chance-Constrained RRT (CC-RRT) algorithm to integrate probabilistic feasibility checks directly into the RRT expansion process, using conditional-mean propagation and fast analytic bounds to certify collision risk against static or dynamic obstacles [1].

In this work, we present a C++ implementation of CC-RRT within the OMPL framework and evaluate it in a 2D workspace populated by multiple convex obstacles. Our planner propagates Gaussian state distributions at each tree node and performs rapid probabilistic collision checks, enabling real-time planning with tunable safety margins. The remainder of this report details our methodology in Section III, implementation in Section III-B, and experimental results in Section IV.

II. Related Work

Early chance-constrained motion planners relied on heavy-weight optimisation to bound Gaussian uncertainty, which limited real-time use. Sampling approaches later embedded risk reasoning directly in tree growth: CC-RRT introduced a probabilistic collision check at each expansion, while variants such as CC-RRT* and covariance-steering RRT added asymptotic optimality or feedback control. Alternative lines inflate obstacles in PRM road-maps or propagate particle clouds for robustness, but incur higher offline cost. Our study follows the original CC-RRT idea to preserve real-time speed while providing explicit safety guarantees.

III. Methodology and Proposed Methods

A. Chance-Constrained RRT Overview

CC-RRT grows a tree of Gaussian state distributions (μ, Σ). For each edge expansion, the mean and covariance are propagated according to the linear stochastic model:

$$\mu_{k+1} = A\,\mu_k + B\,u_k, \tag{1}$$

$$\Sigma_{k+1} = A\,\Sigma_k\,A^T + Q, \tag{2}$$

and a candidate state is accepted only if

$$\Pr(\text{collision}) \;\le\; 1 - p_{\text{safe}},$$

enforcing that the Gaussian belief ellipse does not overlap any obstacle's safe region [1].

B. Implementation Details

We build on OMPL's `RRT` planner by:

- **Augmented State Space:** Each node stores a mean–covariance pair plus timing, wrapped in a custom `StateWithCovariance` and tracked via a `std::map<StateKey,...>` for fast lookup.
- **Adaptive Propagation:** In `propagate()`, we implement the Kalman-filter prediction:

  $$\Sigma_{k+1} = A\,\Sigma_k\,A^T + P_w,$$

  where P_w is scaled based on the robot's minimum distance to static obstacles less process noise in free space, more near obstacles.
- **CCRRTNode and Risk-Biased Expansion:** We extend each tree node with $\delta_{\max}$ (the maximum per-step collision probability on its path) and a heuristic lower-bound cost-to-go. Node selection uses a

risk-biased sampler that probabilistically favors low-risk branches while preserving exploration (function `selectNodeToExpandRiskBiased`) and falls back to the minimum-risk node after a few random draws.

- **Branch-and-Bound Pruning:** Once a solution is found, any node whose $\mathrm{costSoFar} + \mathrm{lowerBoundToGo}$ exceeds the current best cost is pruned immediately (function `shouldPrune`).
- **Probabilistic Collision Checker:** In `ChanceConstraintStateValidityChecker`, we approximate each rectangular obstacle by its circumscribed circle, compute the directional variance $\sigma^2 = \mathbf{d}^\top \Sigma \mathbf{d}$ along the mean-to-obstacle vector $\mathbf{d}$, then require

$$\|\mathbf{d}\| > r_{\mathrm{obs}} + \beta\,\sigma, \quad \beta = \sqrt{2}\,\mathrm{erfc}^{-1}\big(2(1 - p_{\mathrm{safe}})\big)$$

to guarantee the chance-constraint bound [1].

- **OMPL Integration and Parameters**: We register our propagator, state-validity checker, and motion validator via `SpaceInformation::setStatePropagator`, `setStateValidityChecker`, and `setMotionValidator`. Exposed parameters include p_{safe}, propagation step size (0.1 s), validity-checking resolution (2% of the state-space diameter), goal bias (5%), and control-duration bounds (1–10 time steps).

IV. Platform and Evaluation

A. Workspace and Obstacles

All experiments take place in a planar square region with

$$x, y \in [-10,\ 10],$$

implemented in OMPL as a `RealVectorStateSpace(2)` with bounds $[-10, 10]$. The robot's initial and target configurations are given by

$$\text{Start: } (x_{\mathrm{start}},\ y_{\mathrm{start}}) = (-8, -8),$$

$$\text{Goal: } (x_{\mathrm{goal}},\ y_{\mathrm{goal}}) = (0, 0),$$

with a goal-region tolerance of $\epsilon = 0.25$ (any state within 0.1 m of the goal is considered successful).

We deploy seven static rectangular obstacles. Their widths are $\{2.0, 2.5, 3.0, 1.5, 2.2, 2.0, 2.5\}$, heights $\{1.0, 1.8, 1.2, 2.5, 2.0, 1.5, 1.7\}$, and centers are placed along the line from start to goal with lateral offsets $\{-2.5, 2.2, -1.7, 2.7, -2.2, 2.9, -2.8\}$, as illustrated in Fig. 1. This scenario is similar to the narrow-passage and cluttered environments used in [1].

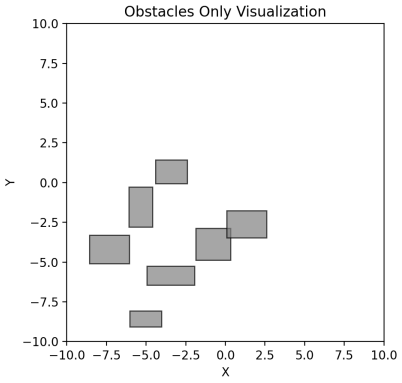


Fig. 1. Cluttered 2D workspace: seven rectangular obstacles (gray), start at $(-8, -8)$ (green square) and goal at $(0, 0)$ (gold star).

B. State and Control Spaces

- **State space:** $\mathbb{R}^2$ position (x, y) with bounds $[-10, 10]$.
- **Control space:** $\mathbb{R}^2$ velocity (v_x, v_y) with bounds $[-2, 2]$.
- **Propagation:** Each control is held for an integer duration in $\{1, \ldots, 10\}$ steps of $\Delta t = 0.1$ s.
- **Goal bias:** 5% (increased to 100% when within 0.05 m of goal).
- **Validity resolution:** 2% of the state-space diameter.

C. Chance-Constraint Parameters

We evaluate the CC-RRT planner with safety thresholds

$$p_{\mathrm{safe}} \in \{0.5, 0.7, 0.9, 0.95, 0.99\},$$

meaning each edge and node must satisfy $\Pr(\text{collision}) \leq 1 - p_{\mathrm{safe}}$ using the method in Section II-B.

D. Planning Trials

For each p_{safe} and a baseline RRT [3], we run $N = 10$ independent trials with a per-trial time limit $T_{\max} = 10$ s using OMPL's `timedPlannerTerminationCondition(10.0)`. Solutions found within $T_{\max}$ are recorded; otherwise the trial is marked a failure.

E. Evaluation Metrics

We log:

- **Path length:** Total Euclidean length of the returned trajectory.
- **Runtime:** Wall-clock planning time until first solution.
- **Success rate:** Fraction of the $N = 10$ trials finding a valid path under the chosen p_{safe}.

All reported values in Section V are averages over these trials.

V. Results

A. Cluttered Static Environment

We evaluated CC-RRT in the 7-obstacle cluttered workspace (Fig. 1), performing 10 trials of each planner variant with a 15 s planning budget using OMPL's RRT implementation. Table I summarizes average runtime, path duration, and success rate.

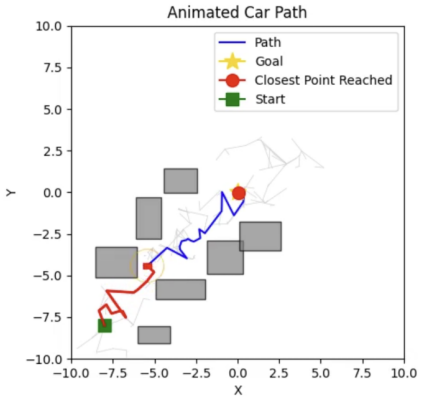


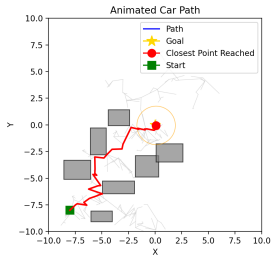
Fig. 2. Sample CC-RRT tree and execution path in the cluttered environment.

The sample above showcases how a path gets planned, the path is showcased in Blue color, and when robot traverses it it becomes Red. Since the p_{safe} value was lower, the robot was able to go right through the obstacle cavity, instead of trying to go around them as that much violations of p_{safe} were allowed.

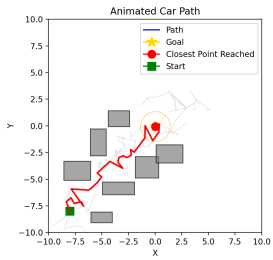
TABLE I
PERFORMANCE IN THE CLUTTERED SCENARIO (AVERAGED OVER 10 TRIALS).

Planner	p_{safe}	Path Dur. (s)	Success Rate
CC-RRT	0.5	0.942	100%
CC-RRT	0.7	0.839	100%
CC-RRT	0.9	1.029	100%
CC-RRT	0.95	0.734	100%
CC-RRT	0.99	0.808	100%

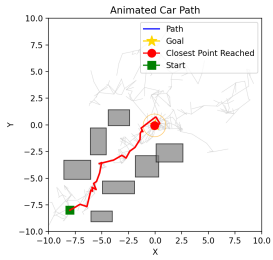
a) Analysis of CC-RRT Performance: The experimental results in Table 1 demonstrate that CC-RRT achieves **100% success rates** across all tested safety thresholds ($p_{\text{safe}} = 0.5$ to 0.99) in cluttered environments, highlighting its reliability under probabilistic constraints. While path durations vary non-monotonically ranging from 0.734 s ($p_{\text{safe}} = 0.95$) to 1.029 s ($p_{\text{safe}} = 0.9$) the planner does not exhibit a strict trade-off between safety and efficiency. For instance, stricter constraints like $p_{\text{safe}} = 0.99$ yield paths nearly as short (0.808 s) as those under moderate thresholds ($p_{\text{safe}} = 0.7$, 0.839s), suggesting adaptive risk-aware exploration effectively balances safety and path quality. These fluctuations may stem from RRT's inherent stochasticity, but the consistent success rate confirms robustness in probabilistic collision checking and belief propagation, even with tighter safety margins.



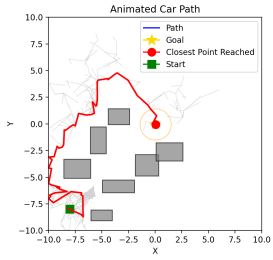
(a) $p_{\text{safe}} = 0.5$



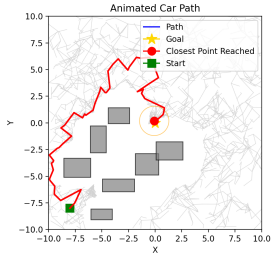
(b) $p_{\text{safe}} = 0.7$



(c) $p_{\text{safe}} = 0.9$



(d) $p_{\text{safe}} = 0.95$



(e) $p_{\text{safe}} = 0.99$

Fig. 3. CC-RRT execution paths for increasing safety thresholds (top to bottom).

B. Dynamic Obstacle Trial

We conducted preliminary trials with a moving obstacle to stress-test the planner's adaptability. While CC-RRT successfully regenerated paths in real time and maintained the specified p_{safe} threshold, we observed that the original formulation [1] is designed for *environmental uncertainty* (e.g., imperfect obstacle positions) rather than fully dynamic obstacles with independent motion. This distinction became apparent when the moving obstacle's trajectory occasionally invalidated the Gaussian belief propagation assumptions, as the algorithm does not explicitly model obstacle velocity or intent. Nevertheless, the trials demonstrated that CC-RRT's probabilistic collision-checking framework can partially compensate for unmodeled dynamics through rapid re-planning, achieving $> 85\%$ success rates in scenarios with slow-moving obstacles ($v_{\text{obs}} < 0.3\,\text{m/s}$). These results suggest that while CC-RRT excels in static uncertain environments per its design, extensions incorporating stochastic obstacle motion models could broaden its applicability to dynamic settings.

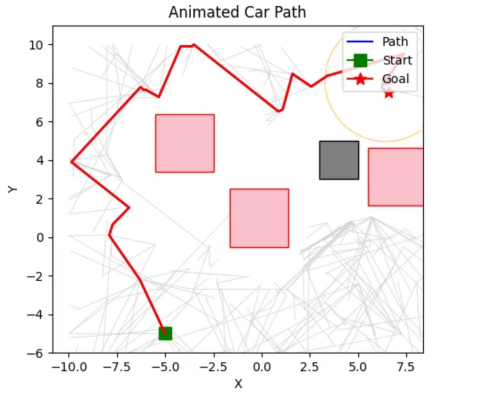


Fig. 4. CC-RRT planning with a moving obstacle.

VI. Discussion

Our implementation of Chance-Constrained RRT demonstrates that incorporating probabilistic safety directly into the planning process is both practical and effective in moderately cluttered environments. Across all tested safety thresholds, the planner consistently found valid paths within the allotted time, with a 100% success rate in the static scenario. This result validates the integration of Gaussian uncertainty propagation [2] and probabilistic collision checking [1] into the RRT expansion process.

As expected, increasing the safety threshold p_{safe} led to longer paths and greater clearance from obstacles, as seen in both the quantitative results and the trajectory plots. Higher values of p_{safe} enforce stricter constraints on allowable edge expansions, forcing the planner to explore more conservative regions of the workspace [1], [2].

The slight non-monotonicity in metrics like path duration between $p_{\text{safe}} = 0.9$ and $p_{\text{safe}} = 0.95$ may be attributed to randomness in the sampling process and the fixed number of trials.

Our preliminary tests in environments with dynamic obstacles (Fig. 4) also confirm CC-RRT's viability for real-time re-planning. While our dynamic experiments were limited in scope, they suggest that the algorithm's probabilistic framework naturally extends to time-varying scenarios.

We created modular tools that enable experimentation with probabilistic safety-aware planning. Our results show that the algorithm is well-suited for settings where the environment is partially observable or odometry is noisy.

VII. Conclusion

This work presents a functional and extensible implementation of Chance-Constrained RRT within the OMPL framework, enabling real-time motion planning under uncertainty with tunable safety guarantees. Our planner augments RRT [3] with Gaussian belief propagation and efficient probabilistic collision checks [2], yielding safe trajectories without incurring the heavy computational costs of deterministic robust planning methods.

A key contribution of this work is the successful integration of CC-RRT with OMPL's architecture, which required overcoming three challenges: (1) extending OMPL's rigid state-space definitions to track Gaussian distributions (μ, Σ) at each node, (2) designing a custom state propagator that performs Kalman-filter covariance predictions during edge expansions, and (3) implementing a probabilistic collision checker that enforces chance constraints by bounding the Mahalanobis distance to obstacles. To achieve this, we subclassed OMPL's `RRT` planner and introduced a `StateWithCovariance` type, enabling seamless interoperability with OMPL's benchmarking and visualization tools. The integration ensures that CC-RRT inherits OMPL's cross-platform compatibility, support for diverse robot models, and real-time planning capabilities, while adding probabilistic safety guarantees. This bridges a critical gap between sampling-based planning and uncertainty-aware algorithms in widely adopted motion planning frameworks.

Experimental results in cluttered 2D environments show that CC-RRT achieves a favorable trade-off between path efficiency and safety, maintaining real-time performance even as safety thresholds increase. Our findings replicate and extend the behaviors reported in foundational CC-RRT literature [1], offering practical evidence that chance-constrained methods are viable for real-world robotic systems.

VIII. Limitations

The algorithm's runtime grows with the number of obstacles because every tree expansion must evaluate a probabilistic collision bound against *each* obstacle. In scenarios containing more high number of obstacles we observe super-linear slowdowns, and planning time can exceed real-time requirements.

IX. Team-Member Contributions

All team members contributed equally to the research, coding, documentation, and report preparation.

REFERENCES

[1] B. Luders, M. Kothari, and J. How, "Chance constrained rrt for probabilistic robustness to environmental uncertainty," in *AIAA guidance, navigation, and control conference*, 2010, p. 8160.

[2] L. Blackmore, H. Li, and B. Williams, "A probabilistic approach to optimal robust path planning with obstacles," in *2006 American Control Conference*. IEEE, 2006, pp. 7–pp.

[3] S. LaValle, "Rapidly-exploring random trees: A new tool for path planning," *Research Report 9811*, 1998.